Computer-Aided IC Design (H05D7)
CAD project 2026:

Sizing analog circuits with reinforcement learning and LLMs

Prof. Georges Gielen
Mohsen Ahmadzadeh, Pierre Graindorge

micas
KU LEUVEN

Outline

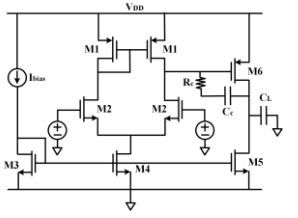
- Goal
- Artificial Neural Networks
- Deep RL for Circuit Sizing
- RL Sizer Setup
- LLM-based Workflows

Outline

- Goal
- Artificial Neural Networks
- Deep RL for Circuit Sizing
- RL Sizer Setup
- LLM-based Workflows

Problem definition

- Develop an AI-based method for automatically sizing an analog circuit
- Target circuit is the two-stage opamp
- The circuit has 20 sizing variables:
  [W_i, L_i, m_i, …, R_c, C_c]
- Consider 6 performance metrics:
  - 4 hard constraints:
    [Gain, Noise, Phase Margin, Unity GBW]
  - 2 optimization targets:
    [Estimated Area, Total Current]

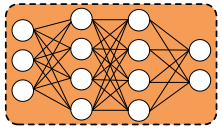


© Georges Gielen, Mohsen Ahmadzadeh, Pierre Graindorge, KU Leuven

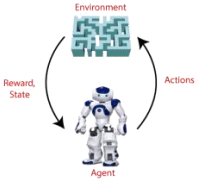
Machine learning tools

- You will learn 3 machine learning concepts:

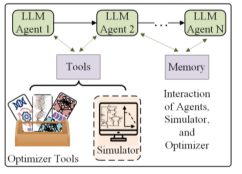
Artificial Neural Networks



Reinforcement Learning



LLM-based Workflows



5

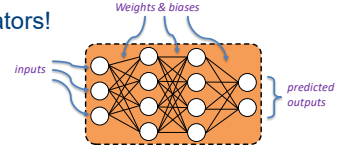
Outline

- Goal
- Artificial Neural Networks
- Deep RL for Circuit Sizing
- RL Sizer Setup
- LLM-based Workflows

6

Artificial neural networks

- ANNs are fancy function approximators!
- They are networks of thousands of randomly initialized parameters (aka weights and biases) that get tuned through exposure to data.



- A Cost Function (or Loss function) is defined to show the accuracy at each step of the tuning.
- Gradient Descent and Error Backpropagation are used to tune those parameters (training/learning) in the direction of *minimizing the cost function*.

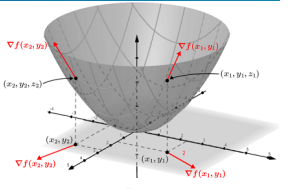
7

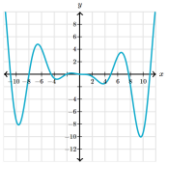
Gradient descent (1)

What is Gradient Descent --> Khan Academy

- The gradient evaluated at any point represents the direction of the steepest ascent
- Consider $f(x) = \frac{x^2 \cos x - x}{10}$
- In Gradient Descent, we gradually change x in the direction of steepest descent; the negative of the gradient.

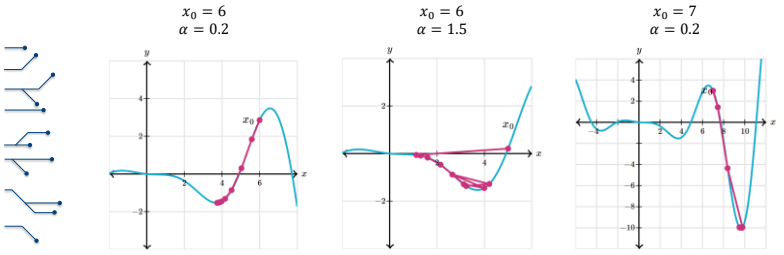
$$x_{n+1} = x_n - \alpha \nabla f(x_n)$$





8

Gradient descent (2)



9

Function approximation (1)

- How can we best approximate $\sin x$ with a degree 5 polynomial within the range $-3 < x < 3$?

$$p(x) = b + w_1 x + \ldots + w_5 x^5$$

- Loss Function to minimize:

$$\mathcal{L}(b, w_1, \ldots, w_5) = \int_{-3}^{3} (p(x) - \sin x)^2 dx$$

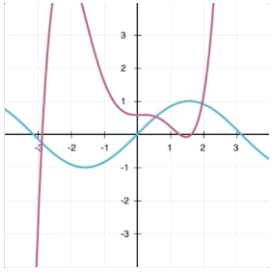
- Our loss is the sum of how incorrect $p(x)$ is at each point
  - → we want to *minimize* this loss with Gradient Descent
  - → we should change weights & biases *in the direction of the negative of the gradient:* $w_i^{n+1} = w_i^n - \alpha \nabla \mathcal{L}(b, w_1, \ldots, w_5)$

10

Function Approximation (2)

- start with $b, w_1, \ldots, w_5$ as random numbers
- *move along the negative gradient, with* $\alpha = 0.001$

$$p(x) = -0.00177 + 0.88974x + 0.00287x^2 - 0.11613x^3 - 0.00048x^4 + 0.00224x^5$$



11

Real-world problems

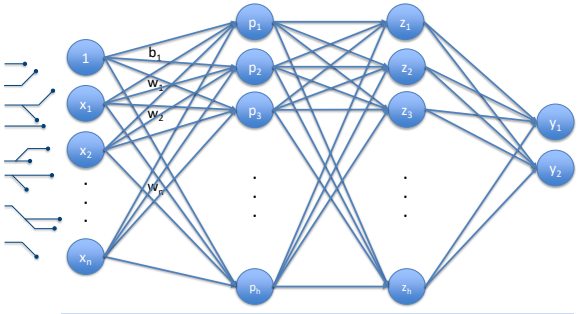
- There are more than 5 features
- The function is more complex (not a simple $\sin x$). So, more neurons and layers are needed.
- Activation function like *sigmoid* and *ReLU* are used by each neuron to add non-linearity.
- The function is a *black box* and expensive to run. We cannot evaluate all points and integrate. Data is hard to collect.
- A *batch* of data is used at each step and their average loss is used for minimization.
- Gradient Descent is applied to weights & biases using the **chain rule** → Error Backpropagation

12

© Georges Gielen, Mohsen Ahmadzadeh, Pierre Graindorge, KU Leuven

Neural networks are here!

What are Neural Networks --> 3Blue1Brown



- Choose a batch of data ($\mathcal{B}$)
- Consider average mean-squared error as loss:

$$\mathcal{L}(b_1, w_1, \dots, w_{100k}) = \frac{1}{\mathcal{B}} \sum_i (Y_i - \hat{Y}_i)^2$$

- Use chain rule to backpropagate the gradient descent updates
- In PyTorch, the neural network is easily written using ***nn.Linear()*** in the ***forward pass***
- Proper loss is chosen based on the algorithm
- An optimizer like ***Adam*** is used with the gradient descent
- Back-propagation is automatically done using ***loss.backward()*** and

❑ hyper-parameters: number of neurons and hidden layers, learning rate α, batch size $\mathcal{B}$, …

13

Outline

- Goal
- Artificial Neural Networks
- Deep RL for Circuit Sizing
- RL Sizer Setup
- LLM-based Workflows

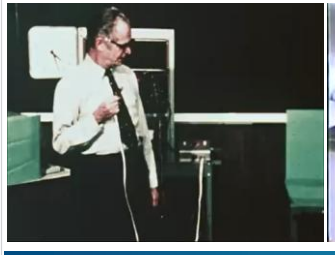
14

Reinforcement !



Free will is an illusion !





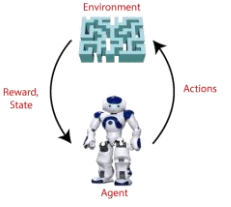


Reinforcement learning

Reinforcement Learning: An Introduction --> Sutton & Barto

Grokking Deep Reinforcement Learning --> Miguel Morales

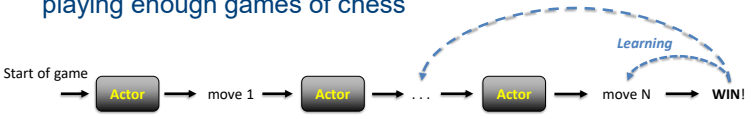
- Learning optimal behavior through interaction with an environment to maximize rewards, like how children learn by exploring
- Uses trial-and-error search to discover best actions without supervision, considering both immediate and delayed rewards
- Key elements of RL:
  1. Agent: The learner or decision-maker.
  2. Environment: Everything the agent interacts with.
  3. State: A specific situation in which the agent finds itself.
  4. Action: All possible moves the agent can make.
  5. Reward: Feedback from the environment based on the action taken.



16

Temporal credit assignment problem

- Start with random moves → give it a reward whenever it won and a punishment whenever it lost → learn by playing enough games of chess

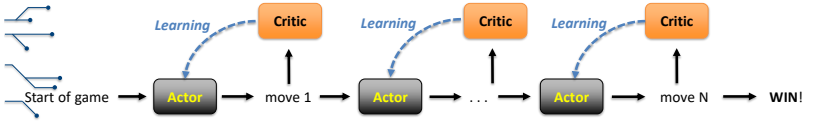


- There are over 10^{120} (!!) possible games of chess (more than the number of atoms in the universe).

17

RL based on temporal differences in expected rewards

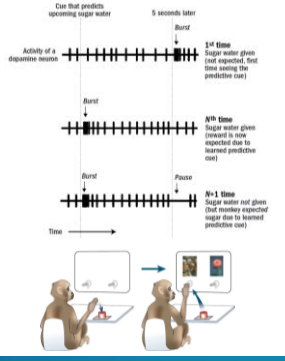
- Learning continuously based on "temporal difference" in the predicted likelihood of winning



18

Temporal difference learning (through dopamine!)

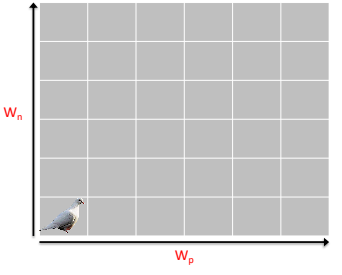
- Wolfram Schultz's monkey experiments
  - showed monkeys different cues (shapes)
  - a few seconds later → sugar water
- At first, dopamine neurons responded after a hungry monkey got sugar water.
- After a few trials:
  - stopped responding to the sugar water itself !!
  - only responded to the predictive cue !!
- Then, not giving them reward after the cue → decreased dopamine response



19

RL for circuit sizing (1)

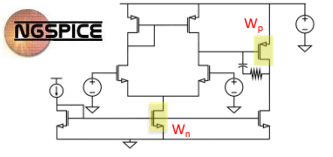
- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} + r_{UGBW}

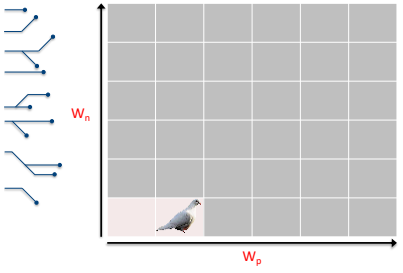
Transition : (state, action, reward, next state)



20

RL for circuit sizing (2)

- Sizing an amplifier in a predictive 45nm CMOS technology

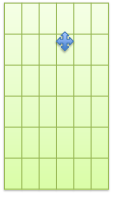


Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain, 0, -1) + r_{UGBW} (UGBW, 0, -1)

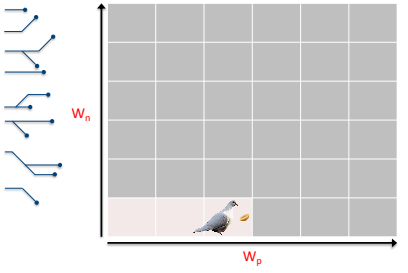
We initialize a state-action-value (Q) table

$Q(s_1, a_1) = 0 + R_2$



RL for circuit sizing (3)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain, 0, -1) + r_{UGBW} (UGBW, 0, -1)

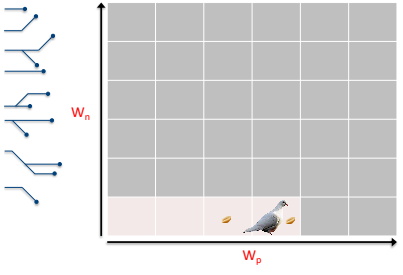
Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3$

$Q(s_2, a_2) = 0 + R_3$

RL for circuit sizing (4)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain, 0, -1) + r_{UGBW} (UGBW, 0, -1)

Finding the value of each action in each state:

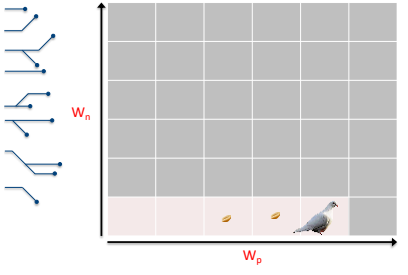
$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4$

$Q(s_3, a_3) = 0 + R_4$

RL for circuit sizing (5)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain, 0, -1) + r_{UGBW} (UGBW, 0, -1)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5$

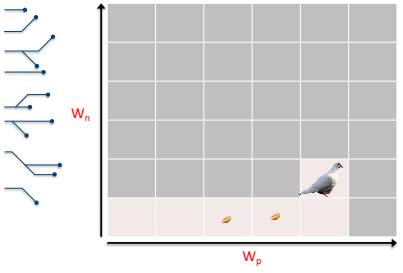
$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5$

$Q(s_4, a_4) = 0 + R_5$

© Georges Gielen, Mohsen Ahmadzadeh, Pierre Graindorge, KU Leuven

RL for circuit sizing (6)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \ldots$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \ldots$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \ldots$

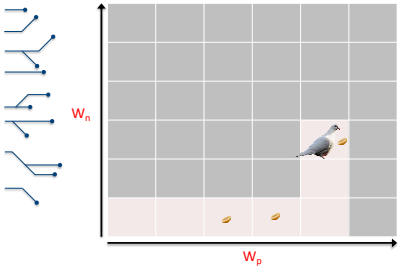
$Q(s_4, a_4) = 0 + R_5 + \ldots$

...

25

RL for circuit sizing (7)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \ldots$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \ldots$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \ldots + \ldots$

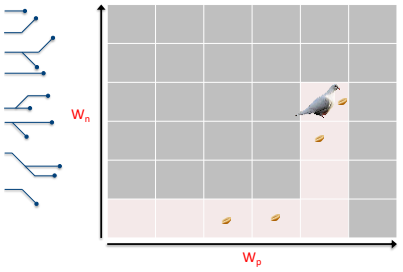
$Q(s_4, a_4) = 0 + R_5 + \ldots + \ldots$

...

26

RL for circuit sizing (8)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \ldots$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \ldots$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \ldots + \ldots$

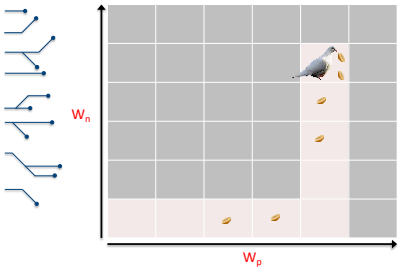
$Q(s_4, a_4) = 0 + R_5 + \ldots + \ldots$

...

27

RL for circuit sizing (9)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \ldots$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \ldots$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \ldots + \ldots$

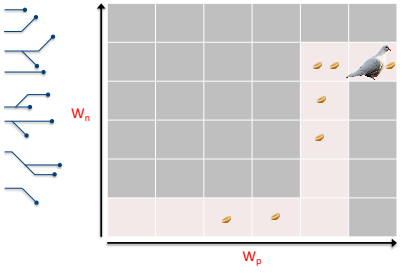
$Q(s_4, a_4) = 0 + R_5 + \ldots + \ldots$

...

28

RL for circuit sizing (10)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \dots$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \dots$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \dots + \dots$

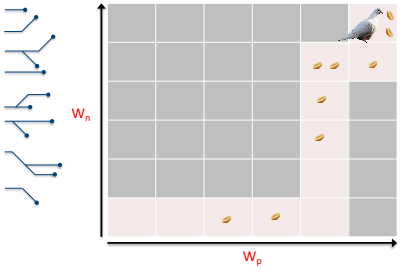
$Q(s_4, a_4) = 0 + R_5 + \dots + \dots + \dots$

...

29

RL for circuit sizing (11)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

Finding the value of each action in each state:

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \dots + \gamma^{t-1}.R_T$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \dots + \gamma^{t-1}.R_T$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \dots + \dots + \gamma^{t-1}.R_T$

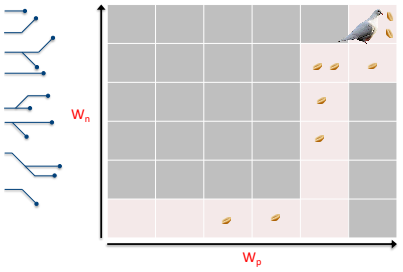
$Q(s_4, a_4) = 0 + R_5 + \dots + \dots + \dots + \gamma^{t-1}.R_T$

...

30

RL for circuit sizing (12)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

$Q(s_1, a_1) = 0 + R_2 + \gamma.R_3 + \gamma^2.R_4 + \gamma^3.R_5 + \dots + \gamma^{t-1}.R_T$

$Q(s_2, a_2) = 0 + R_3 + \gamma.R_4 + \gamma^2.R_5 + \dots + \gamma^{t-1}.R_T$

$Q(s_3, a_3) = 0 + R_4 + \gamma.R_5 + \dots + \dots + \gamma^{t-1}.R_T$

$Q(s_4, a_4) = 0 + R_5 + \dots + \dots + \dots + \gamma^{t-1}.R_T$

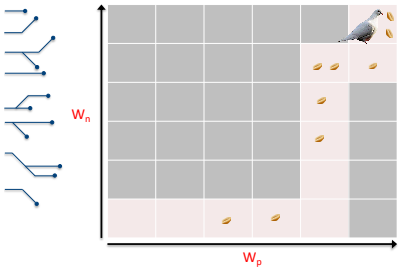
...

$Q(s, a) = R + \gamma.Q(s', a')$

31

RL for circuit sizing (13)

- Sizing an amplifier in a predictive 45nm CMOS technology



Gain > 200
UGBW > 1 MHz

Reward = r_{Gain} (Gain) + r_{UGBW} (UGBW)

$Q(s, a) = R + \gamma.Q(s', a')$

What if we don't want to wait until the end of the Episode?

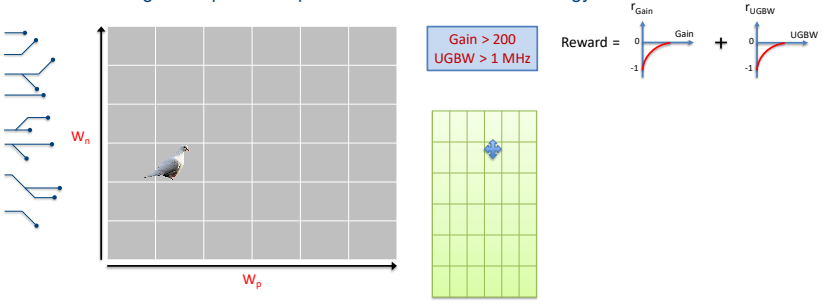
Value of action ***a*** in state ***s*** :

$Q(s, a) = R + \gamma * \max_{a'} Q_{old}(s', a')$

32

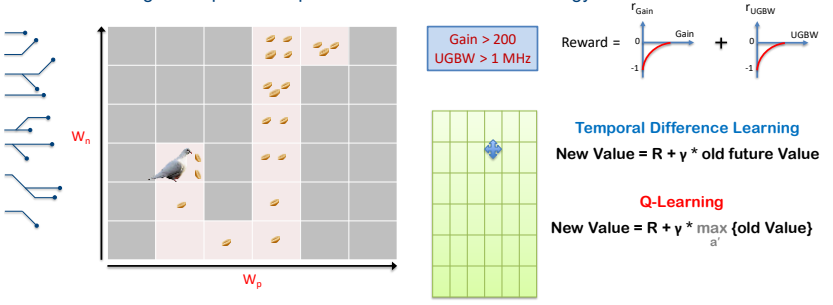
RL for circuit sizing (14)

- Sizing an amplifier in a predictive 45nm CMOS technology



RL for circuit sizing (15)

- Sizing an amplifier in a predictive 45nm CMOS technology



Temporal Difference Learning

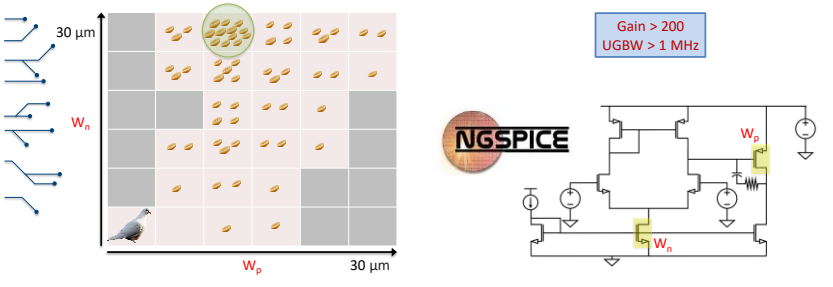
New Value = R + γ * old future Value

Q-Learning

New Value = R + γ * max {old Value}
a'

RL for circuit sizing (16)

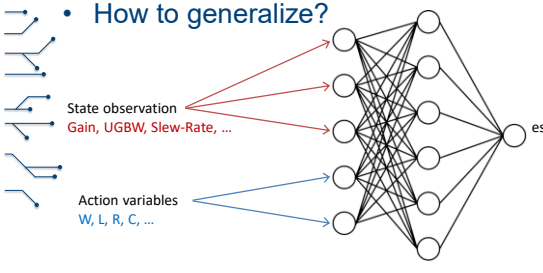
- Sizing an amplifier in a predictive 45nm CMOS technology



Deep Q-learning (1)

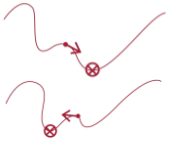
- What about unexplored states?
- How to generalize?

Target ?



Temporal Difference Q-Learning

$q^* = R + \gamma \times \max_{a'} \{Q(s', a')\}$

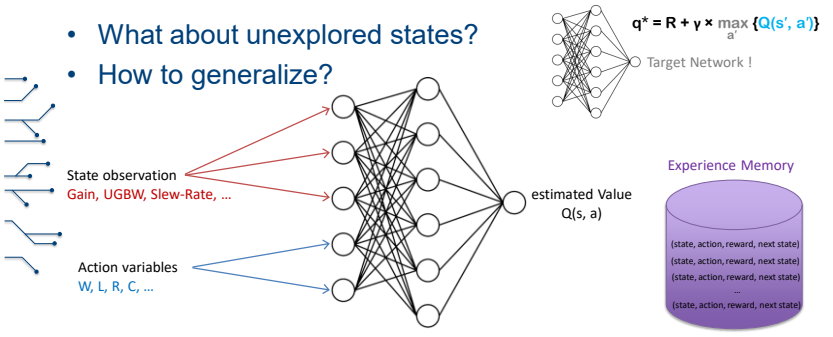


© Georges Gielen, Mohsen Ahmadzadeh, Pierre Graindorge, KU Leuven

Deep Q-learning (2)

Human-level control through deep reinforcement learning --> nature

- What about unexplored states?
- How to generalize?



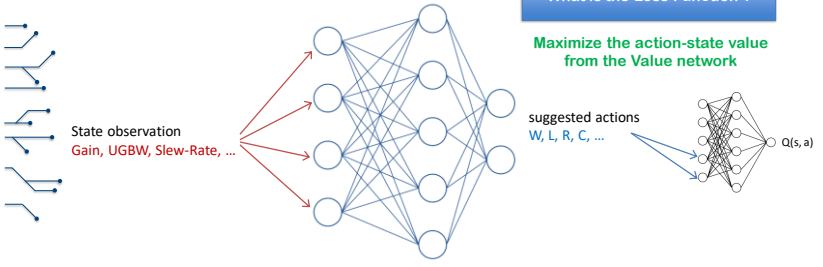
37

Policy gradients

- What about actions?

What is the Loss Function ?

Maximize the action-state value from the Value network

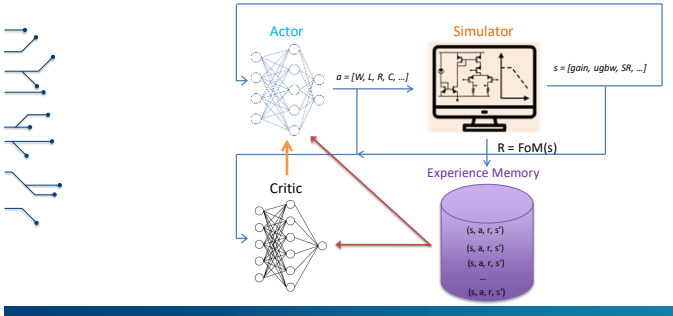


38

Actor-Critic is here!

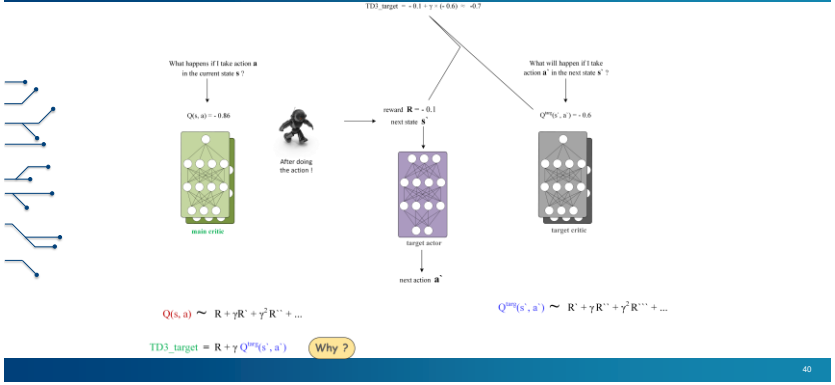
Continuous control with deep reinforcement learning

- Deep Deterministic Policy Gradients (DDPG)



39

TD3 target

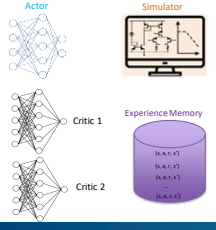


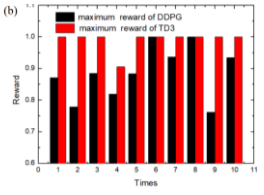
40

More robust Actor-Critic RL

Addressing Function Approximation Error in Actor-Critic Methods

- Twin-Delayed Deep Deterministic Policy Gradients (TD3)
- When using the action with maximum estimated Q-value :
  - → Overestimation bias

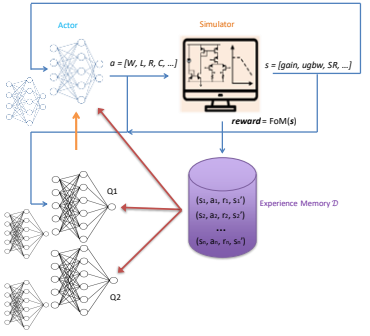




41

TD3 algorithm — Overview

- **Initialize:** Actor, twin critics, target networks, and replay buffer.
- **Collect Experience:** Select actions, simulate the circuit, calculate rewards, and store transitions.
- **Compute Targets:** Sample experiences and calculate target Q-values using target networks.
- **Update Networks:** Train both critics and periodically update the actor to maximize the est imated Q-value.
- **SoftUpdate & Repeat:** Update target networks using Polyak averaging and continue interacting with the simulator.



42

TD3 algorithm — Pseudocode

hyper-parameters
- hidden layer size,
- learning rates α and β
- batch size B
- Warmup steps W
- Episode length
- Action noise σ

```
# Twin Delayed Deep Deterministic Policy Gradients (TD3)
# Hyper-parameters: hidden size, actor lr α, critic lr β, batch size B, warmup steps W, episode
length T, policy noise clip c, policy delay d, discount γ, Polyak τ

1. INPUTS:
    Initialize actor parameters θ  (policy network μ_θ(s))
    Initialize critic parameters φ_1, φ_2 (Q-networks Q_φ1(s, a), Q_φ2(s, a))
    Initialize replay buffer D

2. INITIALIZATION:
    θ_targ ← θ                          # target actor parameters
    φ_targ,1 ← φ_1                      # target critic 1 parameters
    φ_targ,2 ← φ_2                      # target critic 2 parameters

3. EPISODE LOOP:
    for each episode do
        Reset environment and obtain initial state s
        for t = 1, ..., T do
            # --- INTERACTION ---
            a ← μ_θ(s) + ε,    ε ~ N(0, σ)          # add exploration noise
            a ← clip(a, a_low, a_high)              # clip action to valid bounds
            Execute a in environment (simulator)
            Observe reward r, next state s', done flag d
            Store (s, a, r, s', d) in replay buffer D
            s ← s'

            if total_steps > W then                 # start learning after warmup
                # --- GRADIENT UPDATE ---
                Sample a batch {(s_i, a_i, r_i, s'_i, d_i)}_{i=1}^B  from D
                # Compute target actions with clipped noise (target policy smoothing)
                ε̃ ~ clip(N(0, σ), -c, c)
                a'_i ← clip(μ_θ_targ(s'_i) + ε̃, a_low, a_high)

                # Compute target Q-values using twin critics (minimization)
                y_i ← r_i + γ(1 − d_i) min_{k=1,2} Q_φ_targ,k (s'_i, a'_i)

                # Update critic networks by minimizing MSE loss
                for k = 1, 2 do
                    L(φ_k) = (1/B) Σ_{i=1}^B (Q_φk(s_i, a_i) − y_i)^2
                    φ_k ← φ_k − β ∇_φk L(φ_k)
                end for

                # Delayed policy update (every d steps)
                if total_steps mod d == 0 then
                    # Update actor using deterministic policy gradient
                    ∇_θ J(θ) ≈ (1/B) Σ_{i=1}^B ∇_a Q_φ1(s_i, a)|_{a=μ_θ(s_i)} ∇_θ μ_θ(s_i)
                    θ ← θ + α ∇_θ J(θ)      # gradient ascent

                    # Update target networks with Polyak averaging
                    for k = 1, 2 do
                        φ_targ,k ← τ φ_k + (1 − τ) φ_targ,k
                    end for
                    θ_targ ← τ θ + (1 − τ) θ_targ
                end if
            end if

            if d is True then   # episode finished
                break
    4. Repeat for next episode
```

43

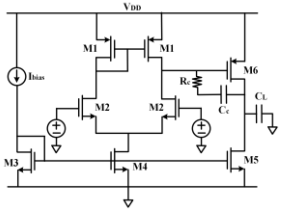
Outline

- Goal
- Artificial Neural Networks
- Deep RL for Circuit Sizing
- RL Sizer Setup
- LLM-based Workflows

44

Two-stage opamp sizing

- Our circuit is a two-stage opamp
- The circuit has 20 sizing variables:
  [W_i, L_i, m_i, …, R_c, C_c]
- We consider 6 performance metrics:
  - 4 hard constraints:
    [Gain, Noise, Phase Margin, Unity GBW]
  - 2 optimization targets:
    [Estimated Area, Total Current]



45

RL elements of circuit sizing

- **FoM (reward):**
  - Satisfy the following specs:
    Gain > 200, Phase Margin > 60 , Unity GBW > 1 *MHz*,
    Slew Rate > *15 V/µs*, Output noise < 30 mV_{rms}
  - While optimizing power and area
  - Bonus reward when all the hard constraints are satisfied
- Action Space:
  a = $[W_1, L_1, m_1, \ldots, W_n, L_n, m_n, R_c, C_c]$
- State Space:
  A vector of the normalized values of the performance metrics (specs)
  $s = [z_1, \ldots, z_y, o_1, \ldots, o_T]$

$$z_y = \frac{y - y^*}{y + y^*} \rightarrow r_y = \begin{cases} \min(z_y, 0), & y \in Y_L \\ -\max(z_y, 0), & y \in Y_U \end{cases}$$

$$r_H = \sum_{y \in Y} r_y$$

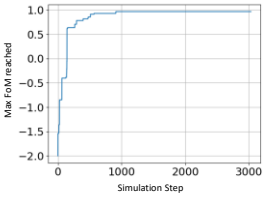
$$o_t = \frac{t - t^*}{t + t^*} \rightarrow r_t = -o_t \rightarrow r_T = \sum_{t \in T} r_t$$

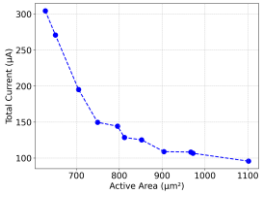
$$FoM = \begin{cases} r_H + 0.05 \times r_T, & r_H < 0 \\ 0.3 + r_T, & r_H \geq 0 \end{cases}$$

46

Outputs and expected results

- TD3 will be used, and we need two plots from you:
  1. Maximum FoM reached per step
  2. Power-Area Pareto Plot of solutions





47

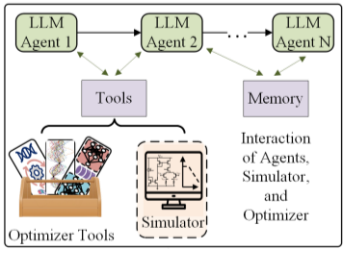
Outline

- Goal
- Artificial Neural Networks
- Deep RL for Circuit Sizing
- RL Sizer Setup
- LLM-based Workflows

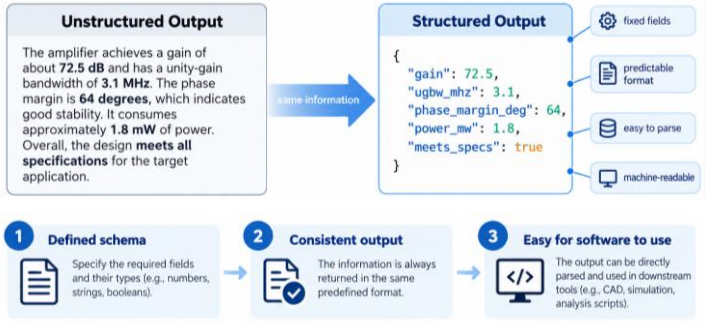
48

LLM-based agentic workflows

- **Era of LLM Agentic AI !**
- LLM workflows have enormous automation and explainability potentials
- Prompts – Context – Skills
- Structured Output – Tools – MCP
- Memory

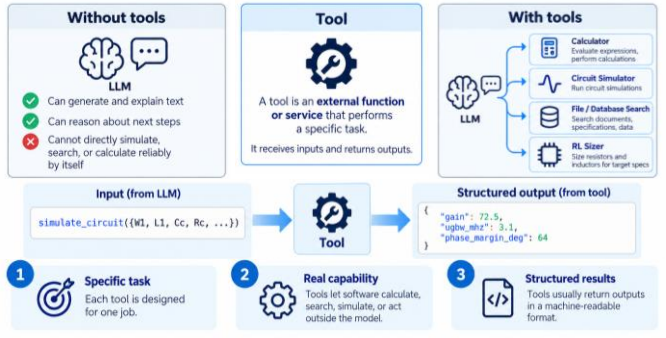


What is structured output?



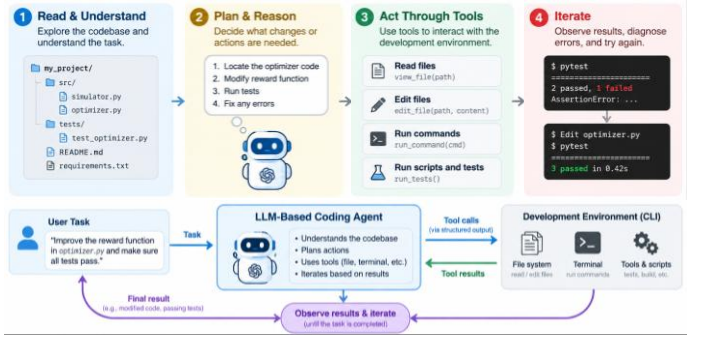
50

What are tools?



51

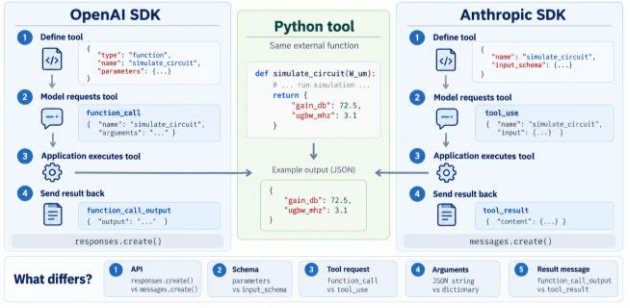
What are LLM-based coding agents ?



52

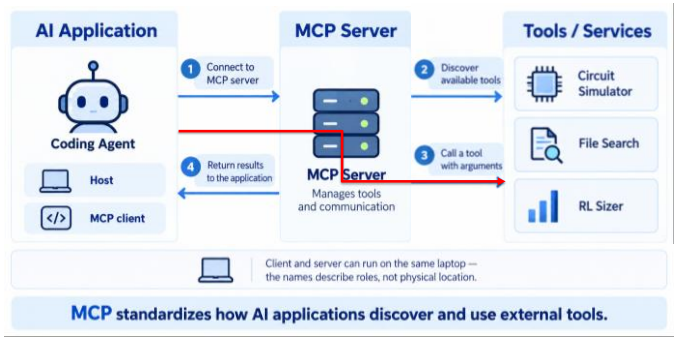
Same tool, Different SDKs!

- OpenAI vs Anthropic tool calling:



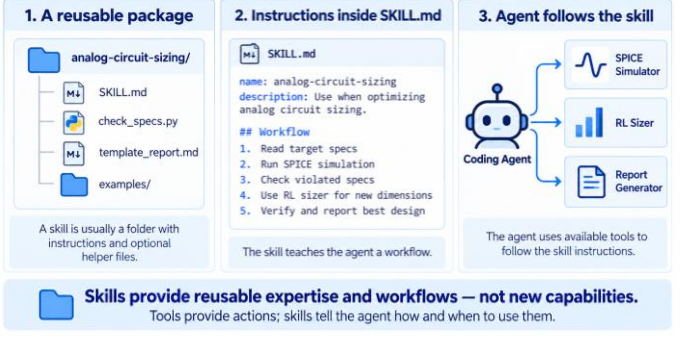
53

What is Model-Context-Protocol (MCP) ?



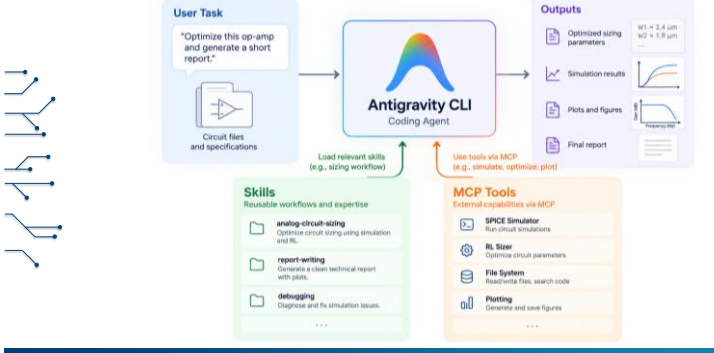
54

What are agent skills?



55

Agentic workflow (Harness)



56

Further learning: some resources

Python
Harvard CS50P
Video course + exercises

Neural Networks
3Blue1Brown: Neural Networks
Excellent visual video series

Reinforcement Learning
Hugging Face Deep RL Course
Hands-on tutorials

Sutton & Barto: Reinforcement Learning: An Introduction
Free textbook

ngspice
Official ngspice Tutorials
Beginner guide + examples

Large Language Models
Andrej Karpathy: Intro to LLMs
1-hour video

MCP
Official MCP Documentation
Introduction + tutorials

Agent Skills
Agent Skills
Standard, examples + documentation

Coding Agents
Claude Code Quickstart
OpenAI Codex Academy
Practical guides

57

PRACTICAL INFORMATION

Sessions – Groups – Report – Toledo

micas KU LEUVEN

Practical information: calendar

5 sessions
2h 25mins each
Nov. 3 – Nov. 6

Deadline: Nov. 8

→ 1 week

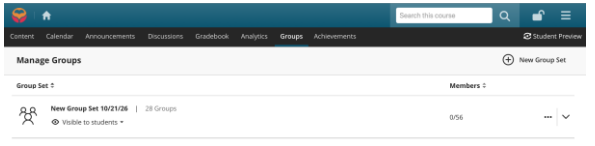


ku leuven timeedit link

59

Practical information: groups

- Groups of **maximum 2 people**
  Self enroll through Toledo before **November 3**



link of groups on ultra page of the course

60

Practical information: final report

- Please attend **in person**, an introduction will be given at the start of each session
- Prepare a final report of **maximum 10 pages**, template below:
  https://www.overleaf.com/latex/templates/ku-leuven-computer-science-template/ymkrfchhfjzw
- The lab project score will account for $^{2}/_{3}$ **of the final score**
- Plagiarism **will not be tolerated**, the **entire content** of the report and of the code can be discussed during the oral exam
- **You can use AI, but we will evaluate YOU!**
- Submit your **final report** and your **code** before: **November 8, 23:59 PM**

61

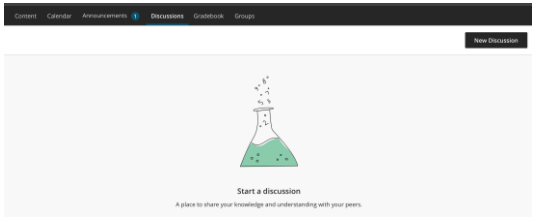
Practical information: exam

- Individual discussion with each student in exam period
  - 2/3 : discussion about your project
  - 1/3 : questions about the course lectures
  - total grade will be rounded to integer score on 20
- Individual timeslot to be sent before the exam
- You can bring all material: your source code, your report, course notes, etc.

62

Practical Information: discussion forum

- Any other question related to the project?
  Please create a (public) discussion through Toledo



https://ultra.edu.kuleuven.cloud/ultra/courses/_168652_1/engagement

63